

Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume V: Practical Application

The Companion Implementation Platform

Preface

Draft Notice. This volume is under active development. Chapter 10 contains a tested three-link prescribed-motion example with explicitly scoped extensions. Other chapters retain preliminary material; the volume does not yet provide a fully implemented or experimentally validated end-to-end golf pipeline.

This is a book about *doing*. Volumes 0–IV developed the theory of motion: from differential geometry through nonlinear control, biomechanics, and motor control. This volume explores how those tools can be connected in computational investigations.

The companion project, UpstreamDrift, is the setting for the software integration discussions in this volume. Engine interfaces and executable examples must be checked against the selected software revision. The capstone here runs the repository’s local three-link model and does not require every engine discussed elsewhere in the volume.

Disclosure. UpstreamDrift is developed by the authors of this textbook. It is not a production simulation engine; it is an educational platform designed to accompany this text and make the theory from Volumes 0–IV executable. The underlying physics computations are delegated to the established engines listed above.

The capstone distinguishes executable model calculations from proposed optimization, muscle, robustness and variability extensions. Its data are synthetic. Each extension needs its own implementation, assumptions and validation before it can support a scientific conclusion.

Contents

Preface	i
Nomenclature	iv
1 The Companion Platform	1
1.1 Architecture Overview	1
1.1.1 Core Components	2
1.1.2 Supported Physics Engines	2
1.2 Getting Started	2
1.3 Engine Selection Guide	5
2 Physics Engine Comparison	6
2.1 Benchmark: Simple Pendulum	6
2.2 Contact Model Comparison	7
2.3 Benchmark Interpretation	8
3 Building a Multi-Body Model	9
3.1 Model Formats	9
3.1.1 URDF (Universal Robot Description Format)	9
3.1.2 MJCF (MuJoCo XML)	9
3.2 Building a 7-Segment Golf Model	9
3.2.1 Programmatic Model Construction	9
4 Forward and Inverse Simulation	13
4.1 Forward Dynamics With the Companion Platform	13
4.2 Simulation Loop and Integration Schemes	15
4.3 Inverse Dynamics Pipeline	15
5 Trajectory Optimization in Practice	16
5.1 DDP/iLQR Implementation	16
5.2 Application: Optimal Pendulum Swing-Up	18
5.3 Application: Optimal Pendulum Swing-Up	19
5.4 Application: Golf Swing Optimization	19
6 Controller Design and Testing	20
6.1 PD Control With Gravity Compensation	20
6.2 Trajectory Tracking With Contraction	21
6.3 Funnel Control	21
7 Parameter Estimation and Model Fitting	22
7.1 System Identification for Multi-Body Models	22
7.2 Parameter Estimation Landscape	23
7.3 Muscle Parameter Calibration	24

8	Reinforcement Learning for Motor Control	25
8.1	RL Environments in the Companion Platform	25
8.2	Agent-Environment Loop	27
8.3	From Gym to Golf	27
9	Visualization and Analysis Tools	28
9.1	3D Motion Visualization	28
9.2	Jacobian and Manipulability Visualization	29
9.3	Energy Flow Diagrams	30
10	Capstone: Golf Swing Analysis	31
10.1	Model and Question	31
10.2	Geometry Before Attribution	31
10.3	What Adds at the Same State	32
10.4	A Counterfactual Is a New Trajectory	33
10.5	A Reproducible Synthetic Example	33
10.6	Connecting Mechanics, Control and Measurement	38

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

These are default conventions; each chapter declares its model, frames, and any local reuse of symbols. The mechanical entries below use n independent, regular local configuration coordinates and $\mathbf{v} = \dot{\mathbf{q}}$. This convention is local: it does not identify every configuration manifold with a Euclidean space or apply unchanged to redundant representations.

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Configuration coordinates in the declared local chart.

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Coordinate rates; the generalized velocities under the local convention above.

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Second time derivatives of these coordinates, not necessarily Cartesian point accelerations.

$\mathbf{x} \in \mathbb{R}^{n_x}$ State-vector representation with n_x components; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ locally, with actuator or internal states added when needed. Redundant state representations also retain their constraints.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces conjugate to $\mathbf{q}$, with power $\boldsymbol{\tau}^T \dot{\mathbf{q}}$; components need not all be joint torques.

$t \in \mathbb{R}$ Time.

More generally, write $\mathbf{q} \in \mathbb{R}^{n_q}$ for a configuration representation and $\mathbf{v} \in \mathbb{R}^{n_v}$ for generalized velocities, with $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\mathbf{v}$ on the admissible domain. A unit quaternion has four components with a norm constraint, while angular velocity has three components in a declared frame. Configuration rates and generalized velocities therefore need not have the same dimension. See [Tedrake's multibody notes](#) for this representation convention. Constraints and any time-dependent kinematics must be included in the chosen model.

Inertia and Dynamics

M(**q**) Mass (inertia) matrix. It is symmetric positive definite for independent regular velocities with strictly positive modeled kinetic energy for every nonzero velocity. Constrained, redundant, massless, or singular representations require their own treatment.

C(**q**, $\dot{\mathbf{q}}$) Coriolis/centrifugal matrix; **C** $\dot{\mathbf{q}}$ is the generalized velocity-product force in the local mechanical equation. The matrix factorization is not unique.

g(**q**) Generalized gravity term; its sign follows the side of the mechanical equation on which the chapter places it.

J(**q**) Coordinate Jacobian of a time-independent task map **p**(**q**), so $\dot{\mathbf{p}} = \mathbf{J}\dot{\mathbf{q}}$ in the stated coordinates and frames.

With $\dot{\mathbf{q}} = \mathbf{N}\mathbf{v}$, the corresponding velocity Jacobian is $\mathbf{J}_v = \mathbf{J}_q\mathbf{N}$. Force components must change with the velocity convention: $\boldsymbol{\tau}_v = \mathbf{N}^T\boldsymbol{\tau}_q$ gives $\boldsymbol{\tau}_v^T\mathbf{v} = \boldsymbol{\tau}_q^T\dot{\mathbf{q}}$. Thus the same club-point motion and mechanical power must result from consistently paired kinematic and force representations.

Control and Optimization

For the indexed entries, declare a discrete update $\mathbf{x}_{k+1} = F_k(\mathbf{x}_k, \mathbf{u}_k)$ and a nominal pair $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$. Its derivatives are evaluated at that pair. These are derivatives of the full update, including the chosen sampling or numerical integration rule, not of zero-input drift alone.

A_{*k*} = $D_x F_k$ State Jacobian of the declared discrete update at the nominal pair.

B_{*k*} = $D_u F_k$ Input Jacobian of that update at the same pair.

V(**x**) or $\mathcal{V}$ Value function or Lyapunov function.

Q State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta\mathbf{u} = -\mathbf{K}\delta\mathbf{x}$).

γ A curve with a declared parameter; for specified initial data and an input law, $\gamma(t)$ denotes a particular timed state trajectory. A solution map $\phi(t; t_0, \mathbf{x}_0)$ instead retains dependence on initial time and state, for a fixed evolution law on its domain of existence.

Continuous-time chapters define their Jacobians from the full differential equation. Some use $f(\mathbf{x}, \mathbf{u}, t)$ for that full right-hand side; the autonomous $f(\mathbf{x})$ below denotes only the declared zero-input drift. Neither convention makes a discrete input Jacobian equal to the continuous one without accounting for the sampling interval and evolution.

Affine Control and Counterfactuals

$f(\mathbf{x})$ Drift vector field: complete autonomous evolution of the declared effective plant when $\mathbf{u} = \mathbf{0}$.

$G(\mathbf{x})$ Input (control) matrix field mapping control inputs to state derivatives.

$\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$ Control-affine system equations of motion.

$\mathbf{x}^{\text{ZTCF}}(t)$ Forward Zero-Torque Counterfactual trajectory under $\mathbf{u}(t) \equiv \mathbf{0}$.

$\mathbf{a}_{\text{ZVCF}}(\mathbf{q}, \mathbf{z})$ Instantaneous Zero-Velocity Counterfactual acceleration with velocity and declared control set to zero.

$\text{DCR}_{W,\mathcal{U}}(\mathbf{x})$ Drift-Control Ratio in a declared acceleration or task-projected space:

$$\text{DCR}_{W,\mathcal{U}}(\mathbf{x}) = \frac{\|W\mathbf{a}_{\text{drift}}\|_2}{\sup_{\mathbf{u} \in \mathcal{U}} \|WB_a\mathbf{u}\|_2 + \varepsilon}.$$

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\text{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

The Companion Platform

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The best theory is one you can run.

1.1 Architecture Overview

UpstreamDrift is the pedagogical platform accompanying this textbook. It provides a unified Python API for multi-body dynamics simulation, trajectory optimization, and motor control analysis by wrapping established physics engines (MuJoCo, Drake, Pinocchio, OpenSim). This chapter provides an overview of the platform’s modular design and engine selection criteria to guide tool choice for specific applications.

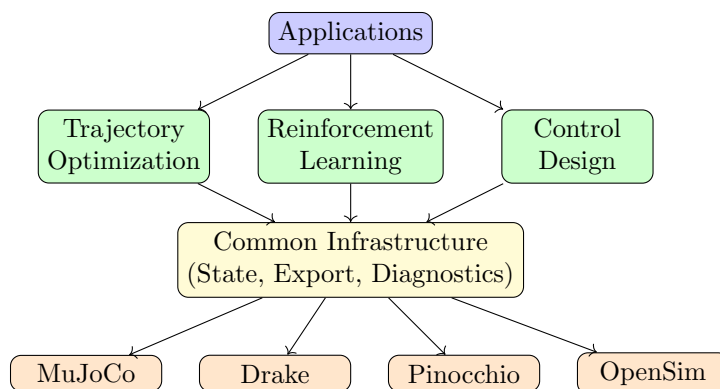


Figure 1.1: UpstreamDrift platform architecture showing the unified Python API (Common Infrastructure layer) sitting above multiple physics engines, with application modules for trajectory optimization, reinforcement learning, and control design.

1.1.1 Core Components

Component	Location	Purpose
Physics Engines	<code>src/engines/physics_engines/</code>	Multi-body simulation
Shared Infrastructure	<code>src/engines/common/</code>	State, export, diagnostics
Pendulum Models	<code>src/engines/pendulum_models/</code>	Simplified test cases
Learning Modules	<code>src/learning/</code>	ML integration
Reinforcement Learning	<code>src/reinforcement_learning/</code>	RL benchmarks
Robotics	<code>src/robotics/</code>	Robot control modules
API	<code>src/api/</code>	External access layer
Tools	<code>src/tools/</code>	Development utilities

1.1.2 Supported Physics Engines

1. **MuJoCo** (`physics_engines/mujoco/`): Multi-body dynamics simulator designed for control and reinforcement learning [19]. Implements soft contact models, supports muscle actuators, and uses efficient convex optimization for contact resolution. Primary engine for biomechanical and contact-rich simulations.
2. **Drake** (`physics_engines/drake/`): Multibody dynamics toolbox from MIT emphasizing mathematical rigor and analytical system properties [18]. Provides analytical Jacobians, integrates trajectory optimization directly, and supports hybrid systems. Primary engine for control design and trajectory optimization.
3. **Pinocchio** (`physics_engines/pinocchio/`): Open-source C++ library (LAAS-CNRS) for rigid-body dynamics with efficient computation of forward and inverse dynamics using the RNEA algorithm [3]. Provides analytical derivatives and real-time performance. Primary engine for real-time control.
4. **OpenSim** (`physics_engines/opensim/`): Musculoskeletal modeling platform from Stanford for biomechanical simulation. Implements Hill-type muscle models and static optimization for muscle force estimation. Primary engine for biomechanics and motion capture analysis.
5. **MyoSuite** (`physics_engines/myosuite/`): Task and environment suite for muscle-driven motor control with OpenSim integration. Provides Gymnasium-compatible interfaces for reinforcement learning. Primary engine for motor learning research.

1.2 Getting Started

```

1 """UpstreamDrift: Getting started with multi-engine simulation.
2
3 This script demonstrates the basic workflow:
4 1. Load a model
5 2. Configure simulation parameters
6 3. Run a simulation

```

```

7 4. Extract and analyze results
8 """
9 import numpy as np
10 from pathlib import Path
11
12 # Configuration
13 MODEL_PATH = Path("src/engines/pendulum_models")
14 ENGINE = "mujoco" # or "drake", "pinocchio"
15
16 def create_simple_pendulum(
17     length: float = 1.0,
18     mass: float = 1.0,
19     damping: float = 0.1
20 ) -> dict:
21     """Create a simple pendulum model configuration.
22
23     Parameters
24     -----
25     length : float
26         Pendulum length in meters.
27     mass : float
28         Bob mass in kilograms.
29     damping : float
30         Joint damping coefficient.
31
32     Returns
33     -----
34     dict
35         Model configuration dictionary.
36     """
37     return {
38         "name": "simple_pendulum",
39         "n_dof": 1,
40         "segments": [{
41             "name": "bob",
42             "mass": mass,
43             "length": length,
44             "com_position": length / 2,
45             "inertia": mass * length**2 / 3,
46         }],
47         "joints": [{
48             "name": "pivot",
49             "type": "revolute",
50             "axis": [0, 0, 1],
51             "damping": damping,
52         }],
53         "gravity": [0, -9.81, 0],
54     }
55
56
57 def simulate_pendulum(
58     config: dict,
59     q0: float = np.pi / 4,
60     t_end: float = 5.0,
61     dt: float = 0.001
62 ) -> dict:
63     """Simulate a simple pendulum using Euler integration.
64
65     Parameters
66     -----

```

```

67     config : dict
68         Model configuration.
69     q0 : float
70         Initial angle (radians from vertical).
71     t_end : float
72         Simulation duration (seconds).
73     dt : float
74         Time step.
75
76     Returns
77     -----
78     dict
79         Simulation results with keys 't', 'q', 'qd', 'energy'.
80     """
81     seg = config["segments"][0]
82     m = seg["mass"]
83     L = seg["length"]
84     b = config["joints"][0]["damping"]
85     g = abs(config["gravity"][1])
86     I = seg["inertia"]
87
88     n_steps = int(t_end / dt)
89     t = np.zeros(n_steps)
90     q = np.zeros(n_steps)
91     qd = np.zeros(n_steps)
92     energy = np.zeros(n_steps)
93
94     q[0] = q0
95     qd[0] = 0.0
96
97     for k in range(n_steps - 1):
98         # Equations of motion:  $I \cdot qdd + b \cdot qd + m \cdot g \cdot L / 2 \cdot \sin(q) = 0$ 
99         qdd = (-b * qd[k] - m * g * L / 2 * np.sin(q[k])) / I
100
101         # Semi-implicit Euler
102         qd[k + 1] = qd[k] + qdd * dt
103         q[k + 1] = q[k] + qd[k + 1] * dt
104         t[k + 1] = t[k] + dt
105
106         # Energy
107         KE = 0.5 * I * qd[k + 1]**2
108         PE = m * g * L / 2 * (1 - np.cos(q[k + 1]))
109         energy[k + 1] = KE + PE
110
111     return {"t": t, "q": q, "qd": qd, "energy": energy}
112
113
114 # Run simulation
115 config = create_simple_pendulum(length=1.0, mass=1.0, damping=0.05)
116 results = simulate_pendulum(config, q0=np.pi/3, t_end=10.0)
117
118 print(f"Simulation complete: {len(results['t'])} timesteps")
119 print(f"Initial energy: {results['energy'][0]:.4f} J")
120 print(f"Final energy: {results['energy'][-1]:.4f} J")
121 print(f"Energy loss: {(1 - results['energy'][-1]/results['energy'][0])
    *100:.1f}%")

```

Listing 1.1: Basic UpstreamDrift usage pattern.

1.3 Engine Selection Guide

Hands-On 1.1. Choosing the Right Engine

Application	Recommended Engine	Key Feature
Fast contact-rich simulation	MuJoCo	Convex soft contact
Trajectory optimization	Drake	Analytical gradients, direct methods
Real-time forward dynamics	Pinocchio	$O(n)$ RNEA algorithm
Muscle-driven simulation	OpenSim	Hill-type muscle models
RL with muscles	MyoSuite	Gym environment wrappers
Analytical gradient-free methods	Built-in Python	No external dependencies

Engine Comparison Details. Table comparisons of contact models, computational complexity, and theoretical guarantees are provided in Chapter 2.

Exercises

1. **First Simulation.** Run the pendulum simulation and plot angle vs. time and energy vs. time. Verify that energy decreases due to damping.
2. **Integration Method.** Replace the semi-implicit Euler with RK4. Compare energy conservation over 100 seconds.
3. **Double Pendulum.** Extend the code to a double pendulum. Plot the chaotic trajectory in configuration space.
4. **Engine Comparison.** Run the same pendulum simulation in MuJoCo and in the built-in Python code. Compare results quantitatively.

Chapter 2

Physics Engine Comparison

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

All models are wrong. Some are useful.
The useful ones depend on the
question.

2.1 Benchmark: Simple Pendulum

We simulate the same pendulum in all engines and compare accuracy, computation time, and energy conservation across different timesteps. The benchmark code provides a repeatable framework for evaluating engine performance on a standard task.

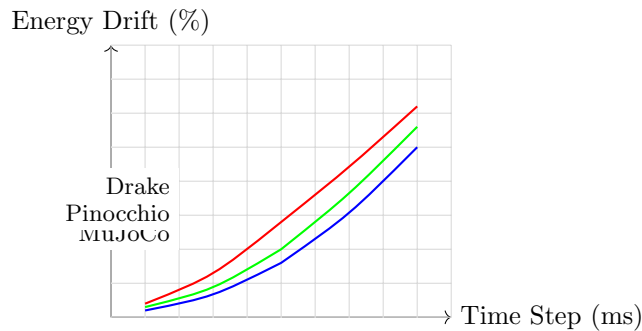


Figure 2.1: Representative energy drift curves for pendulum simulation across physics engines at different timesteps. Smaller drift indicates better numerical integration accuracy. This is a schematic only; actual values depend on integration method and model parameters.

```
1 import numpy as np
2 from dataclasses import dataclass
3 from typing import Protocol
4
5 @dataclass
```

```

6 class BenchmarkResult:
7     """Results from a physics engine benchmark."""
8     engine_name: str
9     positions: np.ndarray      # (n_steps, n_dof)
10    velocities: np.ndarray     # (n_steps, n_dof)
11    times: np.ndarray          # (n_steps,)
12    wall_time_seconds: float
13    energy_drift: float        # relative energy change
14
15 class PhysicsEngine(Protocol):
16     def simulate(self, q0: np.ndarray, qd0: np.ndarray,
17                 t_end: float, dt: float) -> BenchmarkResult: ...
18
19 def compare_engines(
20     engines: dict[str, PhysicsEngine],
21     q0: np.ndarray,
22     t_end: float = 5.0,
23     dt: float = 0.001
24 ) -> dict[str, BenchmarkResult]:
25     """Run the same simulation across multiple engines.
26
27     Parameters
28     -----
29     engines : dict
30         Mapping of engine name to engine instance.
31     q0 : np.ndarray
32         Initial configuration.
33     t_end : float
34         Simulation duration.
35     dt : float
36         Time step.
37
38     Returns
39     -----
40     dict
41         Mapping of engine name to BenchmarkResult.
42     """
43     results = {}
44     qd0 = np.zeros_like(q0)
45
46     for name, engine in engines.items():
47         result = engine.simulate(q0, qd0, t_end, dt)
48         results[name] = result
49         print(f"{name:15s}: wall_time={result.wall_time_seconds:.3f}s, "
50               f"energy_drift={result.energy_drift:.2e}")
51
52     return results

```

Listing 2.1: Cross-engine benchmark framework.

2.2 Contact Model Comparison

Different physics engines implement different contact models, each with trade-offs between computational efficiency and physical accuracy. Table 2.1 summarizes the key approaches.

Engine	Contact Model	Advantages	Limitations
MuJoCo	Soft contact with implicit integration [19]	Fast, stable, convex optimization	Allows non-physical penetration
Drake	Rigid with hydroelastic contact	Physics-based, predictable	Computationally expensive
Pinocchio	No built-in contact handling	Pure rigid-body dynamics	Requires external contact library
OpenSim	Hunt-Crossley damping model	Suitable for biomechanics	Limited geometry types

Table 2.1: Contact model implementations across physics engines. Selection depends on simulation accuracy requirements versus real-time performance constraints.

2.3 Benchmark Interpretation

Benchmark results should be interpreted as follows:

- **Wall Time:** Real computation time (engine-dependent; use for performance comparison only on identical hardware).
- **Energy Drift:** Magnitude of energy conservation error, reported as a percentage. Systems with damping dissipate energy; drift measures additional numerical errors relative to expected dissipation.
- **Trajectory Error:** Point-wise difference between engines in phase space, measured as RMS error or maximum deviation. Serves as a proxy for integration accuracy.

Exercises

1. **Double Pendulum Benchmark.** Run the benchmark framework for a double pendulum across all available engines. Plot wall time versus integration error for each engine.
2. **Energy Conservation Study.** Compare energy conservation across engines with timesteps $\Delta t \in \{0.001, 0.005, 0.01\}$ seconds. Which engine exhibits the smallest energy drift for a fixed timestep?
3. **Contact Model Validation.** Implement a bouncing ball benchmark and compare the computed trajectory with analytical predictions (assuming ideal elastic collision).

Chapter 3

Building a Multi-Body Model

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The model is the hypothesis made physical.

3.1 Model Formats

Multi-body models are typically stored in structured XML formats. Two primary formats are widely supported:

3.1.1 URDF (Universal Robot Description Format)

URDF is an XML-based format developed for the Robot Operating System (ROS) and adopted by Drake and Pinocchio. It defines the kinematic and dynamic structure through `<link>` and `<joint>` elements, along with visual and collision geometry. URDF models use the Denavit-Hartenberg convention or explicit spatial transforms [1].

3.1.2 MJCF (MuJoCo XML)

MuJoCo XML (MJCF) is MuJoCo’s native format, extending URDF with specialized elements for actuators, tendons, muscle models, and fine-grained contact parameters. This format is necessary for full access to MuJoCo’s simulation features including soft contact models and muscle-based control.

3.2 Building a 7-Segment Golf Model

3.2.1 Programmatic Model Construction

Models can be constructed programmatically to enable parameterized design studies. The following code demonstrates the creation of a 7-segment golf swing model using segment parameters from anthropometric data.

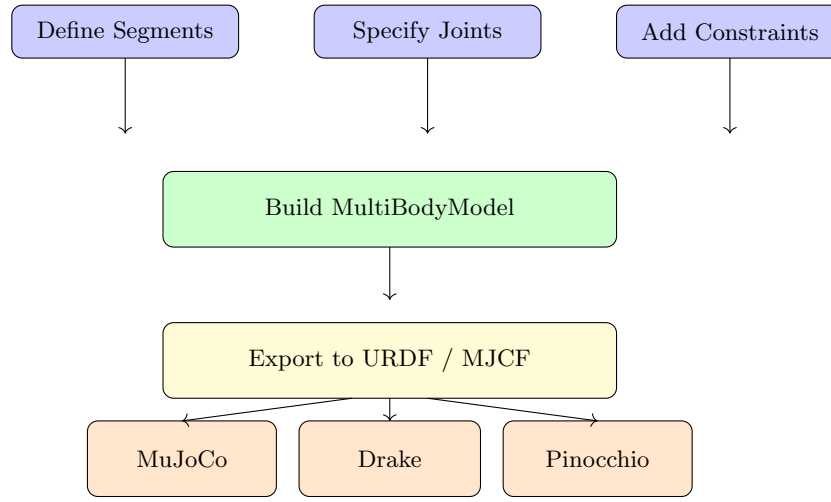


Figure 3.1: Model building workflow: segment definition, joint specification, constraint addition, and export to physics engine formats.

```

1 import numpy as np
2 from dataclasses import dataclass, field
3
4 @dataclass
5 class Link:
6     name: str
7     mass: float
8     length: float
9     com_position: float
10    inertia: np.ndarray = field(default_factory=lambda: np.eye(3))
11
12 @dataclass
13 class Joint:
14     name: str
15     parent: str
16     child: str
17     joint_type: str # revolute, ball, fixed
18     axis: list[float] = field(default_factory=lambda: [0, 0, 1])
19     limits: tuple[float, float] = (-np.pi, np.pi)
20
21 @dataclass
22 class MultiBodyModel:
23     name: str
24     links: list[Link] = field(default_factory=list)
25     joints: list[Joint] = field(default_factory=list)
26
27     @property
28     def n_dof(self) -> int:
29         dof_map = {"revolute": 1, "ball": 3, "fixed": 0}
30         return sum(dof_map.get(j.joint_type, 1) for j in self.joints)
31
32     def to_urdf(self) -> str:
33         """Export model as URDF XML string."""
34         lines = [f'<robot name="{self.name}">']
35         for link in self.links:

```

```

36         lines.append(f' <link name="{link.name}">')
37         lines.append(f' <inertial>')
38         lines.append(f' <mass value="{link.mass}"/>')
39         lines.append(f' </inertial>')
40         lines.append(f' </link>')
41     for joint in self.joints:
42         lines.append(f' <joint name="{joint.name}" '
43                     f'type="{joint.joint_type}">')
44         lines.append(f' <parent link="{joint.parent}"/>')
45         lines.append(f' <child link="{joint.child}"/>')
46         lines.append(f' </joint>')
47     lines.append('</robot>')
48     return '\n'.join(lines)
49
50
51 def build_golf_swing_model() -> MultiBodyModel:
52     """Build a 7-segment golf swing model.
53
54     Segments: pelvis, trunk, upper_arm, forearm, hand, club_shaft, club_head
55     """
56     model = MultiBodyModel(name="golf_swing_7seg")
57
58     # Segment properties from anthropometric tables (80 kg male)
59     # Based on standard biomechanical models from literature
60     model.links = [
61         Link("pelvis", mass=11.2, length=0.15, com_position=0.075),
62         Link("trunk", mass=25.5, length=0.48, com_position=0.22),
63         Link("upper_arm", mass=2.16, length=0.28, com_position=0.16),
64         Link("forearm", mass=1.30, length=0.27, com_position=0.12),
65         Link("hand", mass=0.49, length=0.19, com_position=0.07),
66         Link("club_shaft", mass=0.10, length=1.05, com_position=0.50),
67         Link("club_head", mass=0.20, length=0.10, com_position=0.05),
68     ]
69
70     model.joints = [
71         Joint("pelvis_rotation", "world", "pelvis", "revolute", [0, 1, 0]),
72         Joint("trunk_flexion", "pelvis", "trunk", "revolute", [0, 0, 1]),
73         Joint("shoulder", "trunk", "upper_arm", "ball"),
74         Joint("elbow", "upper_arm", "forearm", "revolute", [0, 0, 1]),
75         Joint("wrist", "forearm", "hand", "revolute", [0, 0, 1]),
76         Joint("grip", "hand", "club_shaft", "fixed"),
77     ]
78
79     return model
80
81
82 model = build_golf_swing_model()
83 print(f"Golf swing model: {len(model.links)} segments, "
84       f"{model.n_dof} DOF")
85 print(f"\nURDF preview:\n{model.to_urdf()[500]}...")

```

Listing 3.1: Programmatic multi-body model construction.

Exercises

1. **Simple Arm Model.** Build a 3-segment arm model (upper arm, forearm, hand) with revolute joints. Define segment masses and lengths, then export as URDF.

2. **Muscle Attachment Points.** Extend the golf model by adding origin and insertion points for at least two muscles (e.g., shoulder and elbow extensors). See Volume III, Chapter 3 for Hill-type muscle models.
3. **Visual Verification.** Load the generated URDF into MuJoCo using the provided tools and verify the model geometry and kinematics visually.

Chapter 4

Forward and Inverse Simulation

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Forward simulation answers “what happens if...?”

Inverse dynamics answers “what caused this?”

The forward dynamics problem computes accelerations from forces, while inverse dynamics (also called the inverse kinematics problem when phrased in terms of finding required torques) computes forces from desired accelerations. Both are fundamental to control design and trajectory planning.

4.1 Forward Dynamics With the Companion Platform

Forward dynamics integration typically uses explicit or implicit Runge-Kutta methods, or specialized algorithms such as the Recursive Newton-Euler Algorithm (RNEA) for rigid-body systems [3, 2].

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 def forward_dynamics_rnea(
5     q: np.ndarray,
6     qd: np.ndarray,
7     tau: np.ndarray,
8     model_params: dict
9 ) -> np.ndarray:
10     """Compute joint accelerations using RNEA-based forward dynamics.
11
12     Parameters
13     -----
14     q : np.ndarray, shape (n,)
15         Joint positions.
16     qd : np.ndarray, shape (n,)
17         Joint velocities.
18     tau : np.ndarray, shape (n,)
19         Applied joint torques.
20     model_params : dict
```

```

21     Model parameters (masses, lengths, inertias).
22
23     Returns
24     -----
25     np.ndarray, shape (n,)
26         Joint accelerations.
27     """
28     n = len(q)
29     masses = model_params['masses']
30     lengths = model_params['lengths']
31     g = model_params.get('gravity', 9.81)
32
33     # Simple 2-link model for demonstration
34     if n == 2:
35         m1, m2 = masses[:2]
36         l1, l2 = lengths[:2]
37         lc1, lc2 = l1/2, l2/2
38         I1 = m1 * l1**2 / 12
39         I2 = m2 * l2**2 / 12
40
41         # Mass matrix
42         M = np.array([
43             [I1 + I2 + m1*lc1**2 + m2*(l1**2 + lc2**2 + 2*l1*lc2*np.cos(q[1])
44             ),
45              [I2 + m2*(lc2**2 + l1*lc2*np.cos(q[1]))],
46              [I2 + m2*(lc2**2 + l1*lc2*np.cos(q[1])),
47              I2 + m2*lc2**2]
48         ])
49
50         # Coriolis
51         h = m2 * l1 * lc2 * np.sin(q[1])
52         C = np.array([[ -h*qd[1], -h*(qd[0]+qd[1])],
53                       [ h*qd[0], 0]])
54
55         # Gravity
56         gvec = np.array([
57             (m1*lc1 + m2*l1)*g*np.sin(q[0]) + m2*lc2*g*np.sin(q[0]+q[1]),
58             m2*lc2*g*np.sin(q[0]+q[1])
59         ])
60
61         qdd = np.linalg.solve(M, tau - C @ qd - gvec)
62     else:
63         # Placeholder for general n-DOF
64         qdd = np.zeros(n)
65
66     return qdd
67
68 # Example simulation
69 params = {'masses': [2.0, 1.0], 'lengths': [0.3, 0.25], 'gravity': 9.81}
70 q0 = np.array([np.pi/4, np.pi/6])
71 qd0 = np.zeros(2)
72 qdd = forward_dynamics_rnea(q0, qd0, np.zeros(2), params)
73 print(f"Gravity-driven accelerations: {qdd}")

```

Listing 4.1: Forward dynamics simulation pipeline.

4.2 Simulation Loop and Integration Schemes

Figure 4.1 illustrates the typical forward simulation loop. At each timestep, we compute accelerations from the equations of motion, integrate to update velocities and positions, and advance to the next timestep.

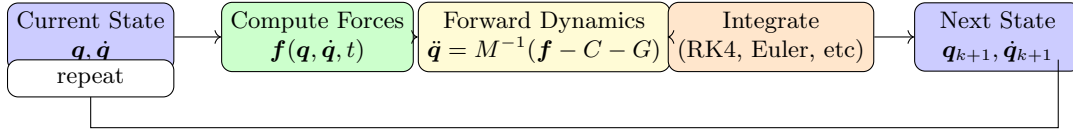


Figure 4.1: Forward simulation loop: from current state to forces (gravity, contacts, actuators), compute accelerations via forward dynamics, integrate, and update to next state.

4.3 Inverse Dynamics Pipeline

The inverse dynamics problem solves for torques given desired motion: given $\mathbf{q}(t), \dot{\mathbf{q}}(t), \ddot{\mathbf{q}}(t)$, compute $\boldsymbol{\tau}(t)$. This is implemented via the Recursive Newton-Euler Algorithm (RNEA) [3, 7]. The RNEA can be run either forward (for forward dynamics) or backward (for inverse dynamics) through the kinematic chain.

Exercises

1. Run forward dynamics for a 2-link pendulum and plot trajectories.
2. Implement inverse dynamics and verify $\text{ID}(\text{FD}(\boldsymbol{\tau})) = \boldsymbol{\tau}$.
3. Compare with Pinocchio's RNEA implementation for timing.

Chapter 5

Trajectory Optimization in Practice

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Theory without practice is empty.
Practice without theory is blind.

5.1 DDP/iLQR Implementation

Differential Dynamic Programming (DDP) and iterative Linear Quadratic Regulator (iLQR) are related algorithms for trajectory optimization [13, 4]. They solve the finite-horizon optimal control problem by iteratively linearizing around a nominal trajectory and solving quadratic subproblems. Figure 5.1 shows the algorithm structure.

Here we implement iLQR end-to-end:

```
1 import numpy as np
2 from typing import Callable, Tuple
3
4 def ilqr_solve(
5     dynamics: Callable,
6     cost_running: Callable,
7     cost_terminal: Callable,
8     x0: np.ndarray,
9     u_init: np.ndarray,
10    n_iterations: int = 50,
11    regularization: float = 1e-6
12 ) -> Tuple[np.ndarray, np.ndarray, list[float]]:
13     """Iterative Linear Quadratic Regulator.
14
15     Parameters
16     -----
17     dynamics : callable
18         (x, u) -> x_next, (A, B) Jacobians
19     cost_running : callable
20         (x, u) -> (cost, lx, lu, lxx, luu, lux)
21     cost_terminal : callable
22         x -> (cost, lx, lxx)
```

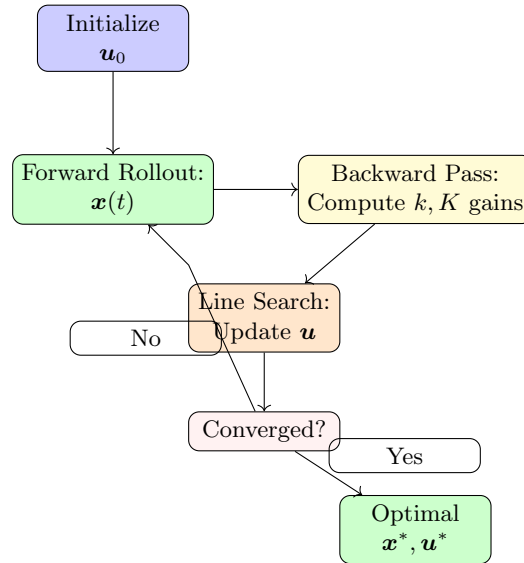


Figure 5.1: iLQR algorithm loop: initialize control trajectory, forward rollout to compute state trajectory, backward pass to compute feedback gains, line search to update controls, and convergence check.

```

23     x0 : np.ndarray
24         Initial state.
25     u_init : np.ndarray, shape (T, m)
26         Initial control trajectory.
27     n_iterations : int
28         Maximum iterations.
29     regularization : float
30         Regularization for Quu inversion.
31
32     Returns
33     -----
34     Tuple
35         (x_traj, u_traj, costs)
36     """
37     T, m = u_init.shape
38     n = len(x0)
39
40     # Forward rollout with initial controls
41     x_traj = np.zeros((T + 1, n))
42     x_traj[0] = x0
43     u_traj = u_init.copy()
44
45     for t in range(T):
46         x_next, _ = dynamics(x_traj[t], u_traj[t])
47         x_traj[t + 1] = x_next
48
49     costs = []
50
51     for iteration in range(n_iterations):
52         # Compute total cost
53         total_cost = sum(

```

```

54         cost_running(x_traj[t], u_traj[t])[0]
55         for t in range(T)
56     ) + cost_terminal(x_traj[T])[0]
57     costs.append(total_cost)
58
59     # Backward pass
60     Vx_next, Vxx_next = cost_terminal(x_traj[T])[1:]
61     k_gains = np.zeros((T, m))
62     K_gains = np.zeros((T, m, n))
63
64     for t in range(T - 1, -1, -1):
65         _, (A, B) = dynamics(x_traj[t], u_traj[t])
66         _, lx, lu, lxx, luu, lux = cost_running(
67             x_traj[t], u_traj[t]
68         )
69
70         Qx = lx + A.T @ Vx_next
71         Qu = lu + B.T @ Vx_next
72         Qxx = lxx + A.T @ Vxx_next @ A
73         Quu = luu + B.T @ Vxx_next @ B + regularization * np.eye(m)
74         Qux = lux + B.T @ Vxx_next @ A
75
76         Quu_inv = np.linalg.inv(Quu)
77         k_gains[t] = -Quu_inv @ Qu
78         K_gains[t] = -Quu_inv @ Qux
79
80         Vx_next = Qx + K_gains[t].T @ Quu @ k_gains[t]
81         Vxx_next = Qxx + K_gains[t].T @ Quu @ K_gains[t]
82
83     # Forward pass with line search
84     alpha = 1.0
85     x_new = np.zeros_like(x_traj)
86     u_new = np.zeros_like(u_traj)
87     x_new[0] = x0
88
89     for t in range(T):
90         dx = x_new[t] - x_traj[t]
91         u_new[t] = u_traj[t] + alpha * k_gains[t] + K_gains[t] @ dx
92         x_new[t + 1], _ = dynamics(x_new[t], u_new[t])
93
94     x_traj = x_new
95     u_traj = u_new
96
97     if iteration > 0 and abs(costs[-1] - costs[-2]) < 1e-6:
98         break
99
100     return x_traj, u_traj, costs

```

Listing 5.1: iLQR implementation for trajectory optimization.

5.2 Application: Optimal Pendulum Swing-Up

The classic control benchmark: swing a pendulum from the hanging position to the inverted equilibrium.

5.3 Application: Optimal Pendulum Swing-Up

The pendulum swing-up is a classic trajectory optimization benchmark. The goal is to find the minimum-energy torque sequence that brings the pendulum from the downward (stable) equilibrium to the upright (unstable) equilibrium.

5.4 Application: Golf Swing Optimization

DDP can be applied to a simplified golf model (3-5 DOF) to optimize clubhead speed at ball impact. The cost function would penalize control effort while maximizing endpoint velocity. In practice, golf swing optimization requires careful handling of contact constraints at impact.

Exercises

1. Use the iLQR code to solve the pendulum swing-up problem.
2. Extend to a double pendulum and optimize for maximum tip speed.
3. Add muscle force constraints and solve the muscle-driven swing-up.

Chapter 6

Controller Design and Testing

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

A controller is only as good as its worst-case performance.

This chapter covers implementation of feedback controllers for multi-body systems, from proportional-derivative (PD) control to contraction-based designs. All controllers assume full state feedback, i.e., direct access to positions and velocities.

6.1 PD Control With Gravity Compensation

The proportional-derivative controller with gravity compensation is the simplest nonlinear controller that guarantees asymptotic stability for mechanical systems without external disturbances [16].

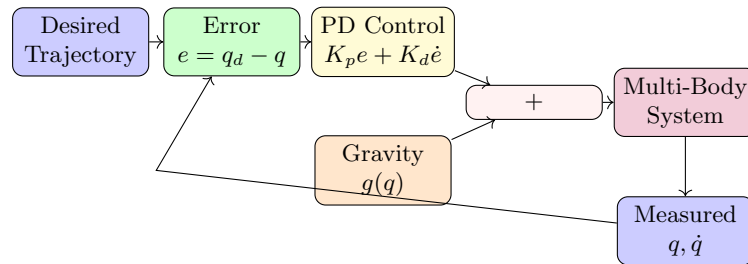


Figure 6.1: PD control with gravity compensation. The controller computes an error signal from desired and measured positions, applies PD gains, and adds gravity compensation to achieve asymptotic tracking.

```
1 import numpy as np
2
3 class PDGravityCompController:
4     """PD controller with gravity compensation.
5
6     From Volume I, Chapter 3: the simplest nonlinear controller
```

```

7     that guarantees asymptotic stability.
8     """
9
10    def __init__(self, Kp: np.ndarray, Kd: np.ndarray,
11                  gravity_func=None):
12        self.Kp = Kp
13        self.Kd = Kd
14        self.gravity_func = gravity_func
15
16    def compute(self, q: np.ndarray, qd: np.ndarray,
17               q_des: np.ndarray, qd_des: np.ndarray) -> np.ndarray:
18        """Compute control torques.
19
20        Parameters
21        -----
22        q, qd : current state
23        q_des, qd_des : desired state
24
25        Returns
26        -----
27        tau : np.ndarray, control torques
28        """
29        tau = self.Kp @ (q_des - q) + self.Kd @ (qd_des - qd)
30        if self.gravity_func is not None:
31            tau += self.gravity_func(q)
32        return tau

```

Listing 6.1: PD controller with gravity compensation.

6.2 Trajectory Tracking With Contraction

Contraction theory provides tools for analyzing nonlinear stability with global guarantees [6, 12]. A contraction-based controller designs the feedback law to make the closed-loop system contracting with respect to a suitable metric, guaranteeing exponential convergence to the desired trajectory from any initial condition.

6.3 Funnel Control

Trajectory funnels define regions around a nominal trajectory within which the controller guarantees convergence [11]. This is useful for handling model uncertainty and disturbances. Funnel computation requires solving a set of linear matrix inequalities (LMIs) at each time step.

Exercises

1. Implement PD control for the 2-DOF arm and track a circular trajectory.
2. Add gravity compensation and compare tracking error.
3. Implement a contraction-based controller and verify convergence.

Chapter 7

Parameter Estimation and Model Fitting

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

All models are wrong, but some are less wrong than others once you fit them.

Parameter estimation uses measured motion data to infer unknown model parameters such as segment masses, inertias, and damping coefficients. This chapter covers least-squares methods for linear and bilinear problems, and optimization methods for nonlinear problems.

7.1 System Identification for Multi-Body Models

The linear regression approach exploits the linearity of the equations of motion with respect to inertial parameters. Given measured trajectories $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, $\ddot{\mathbf{q}}(t)$ and applied torques $\boldsymbol{\tau}(t)$, we form the regressor matrix $Y(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$ such that

$$\boldsymbol{\tau} = Y(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})\boldsymbol{\pi} \quad (7.1)$$

where $\boldsymbol{\pi}$ is the parameter vector [7]. The least-squares solution minimizes $\|Y\boldsymbol{\pi} - \boldsymbol{\tau}\|_2^2$.

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def estimate_inertial_params(
5     q_data: np.ndarray,
6     qd_data: np.ndarray,
7     qdd_data: np.ndarray,
8     tau_data: np.ndarray,
9     n_params: int
10 ) -> np.ndarray:
11     """Estimate inertial parameters using least-squares.
12
13     The equations of motion are LINEAR in inertial parameters:
14     tau = Y(q, qd, qdd) * pi
15     where Y is the regressor matrix and pi is the parameter vector.
```

```

16
17     Parameters
18     -----
19     q_data : (N, n) joint positions
20     qd_data : (N, n) joint velocities
21     qdd_data : (N, n) joint accelerations
22     tau_data : (N, n) joint torques
23     n_params : int number of inertial parameters
24
25     Returns
26     -----
27     np.ndarray estimated parameters
28     """
29     N, n = q_data.shape
30
31     # Build regressor matrix (simplified for 1-DOF)
32     Y = np.zeros((N * n, n_params))
33     tau_vec = tau_data.reshape(-1)
34
35     for k in range(N):
36         for j in range(n):
37             row = k * n + j
38             Y[row, 0] = qdd_data[k, j] # Inertia
39             Y[row, 1] = np.sin(q_data[k, j]) # Gravity
40
41     # Least squares solution
42     params, _, _, _ = np.linalg.lstsq(Y, tau_vec, rcond=None)
43     return params

```

Listing 7.1: Inertial parameter estimation from motion data.

7.2 Parameter Estimation Landscape

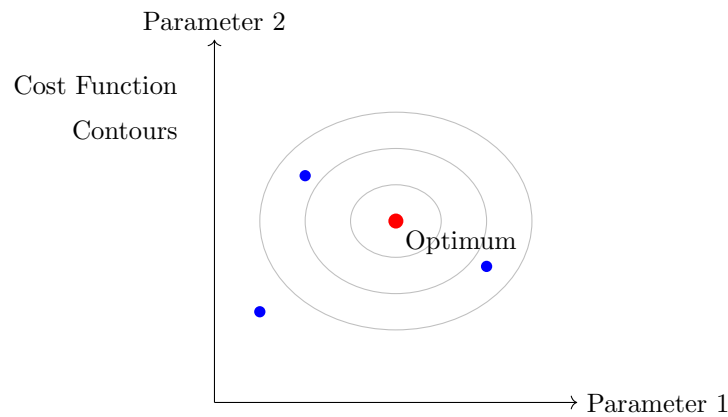


Figure 7.1: Parameter estimation as optimization over the parameter space. Least-squares methods seek the minimum of the cost function (red point) from initial guesses (blue points).

7.3 Muscle Parameter Calibration

Muscle parameters (Hill model constants: a , $F_{\max}$, optimal fiber length, etc.) can be estimated by fitting model predictions to force or EMG measurements. This requires nonlinear optimization, typically using gradient-based methods [14].

Exercises

1. Estimate the mass and length of a pendulum from simulated motion data.
2. Add measurement noise and study estimation robustness.
3. Implement Bayesian estimation with prior knowledge from literature.

Chapter 8

Reinforcement Learning for Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

When the model is too complex for optimal control, let the agent learn.

Reinforcement learning (RL) methods can solve motor control problems when the cost function is difficult to specify or when model uncertainty is large [17]. This chapter demonstrates how to formulate multi-body control tasks as RL environments and integrate with standard policy optimization algorithms like PPO [15].

8.1 RL Environments in the Companion Platform

The companion platform provides Gymnasium-compatible environments for motor control tasks [17]. Gymnasium (formerly OpenAI Gym) is a standard interface for defining RL environments with reset, step, and reward computation methods. This enables use of any RL algorithm implemented for Gymnasium.

```
1 import numpy as np
2
3 class PendulumEnv:
4     """Simple pendulum environment for RL benchmarks.
5
6     State: [cos(theta), sin(theta), theta_dot]
7     Action: torque (continuous, clipped to [-2, 2])
8     Reward: -(angle^2 + 0.1*vel^2 + 0.01*action^2)
9     """
10
11     def __init__(self, dt: float = 0.05, max_steps: int = 200):
12         self.dt = dt
13         self.max_steps = max_steps
14         self.g = 9.81
15         self.l = 1.0
16         self.m = 1.0
17         self.max_torque = 2.0
18         self.state = np.zeros(2)
```

```

19     self.step_count = 0
20
21     def reset(self, seed: int | None = None) -> np.ndarray:
22         if seed is not None:
23             np.random.seed(seed)
24         self.state = np.array([
25             np.pi + np.random.uniform(-0.1, 0.1),
26             np.random.uniform(-0.5, 0.5)
27         ])
28         self.step_count = 0
29         return self._get_obs()
30
31     def _get_obs(self) -> np.ndarray:
32         theta, theta_dot = self.state
33         return np.array([np.cos(theta), np.sin(theta), theta_dot])
34
35     def step(self, action: float) -> tuple:
36         theta, theta_dot = self.state
37         u = np.clip(action, -self.max_torque, self.max_torque)
38
39         # Dynamics
40         theta_ddot = (-self.g / self.l * np.sin(theta) +
41                     u / (self.m * self.l**2))
42         theta_dot += theta_ddot * self.dt
43         theta += theta_dot * self.dt
44
45         self.state = np.array([theta, theta_dot])
46         self.step_count += 1
47
48         # Reward
49         angle_normalized = ((theta + np.pi) % (2*np.pi)) - np.pi
50         reward = -(angle_normalized**2 +
51                 0.1 * theta_dot**2 +
52                 0.01 * u**2)
53
54         done = self.step_count >= self.max_steps
55         return self._get_obs(), reward, done, {}
56
57
58 # Quick test
59 env = PendulumEnv()
60 obs = env.reset(seed=42)
61 total_reward = 0.0
62 for _ in range(200):
63     action = np.random.uniform(-2, 2)
64     obs, reward, done, _ = env.step(action)
65     total_reward += reward
66     if done:
67         break
68 print(f"Random policy total reward: {total_reward:.1f}")

```

Listing 8.1: Custom RL environment for pendulum control.

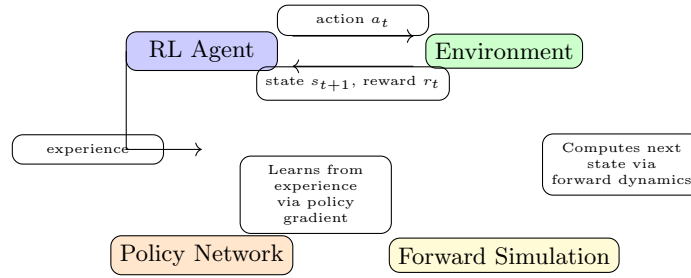


Figure 8.1: RL agent-environment loop. The agent selects actions via its policy network; the environment integrates dynamics and computes rewards. Both interact repeatedly.

8.2 Agent-Environment Loop

8.3 From Gym to Golf

The companion platform’s RL pipeline extends from simple benchmarks (pendulum, reaching) to full muscle-driven golf swing optimization using MyoSuite environments. The reward function is task-specific; for golf, one might maximize clubhead speed at impact while penalizing muscle effort.

Exercises

1. Train a PPO agent to swing up the pendulum. Plot the learning curve.
2. Compare RL-learned policy with the iLQR solution from Chapter 5.
3. Design a reward function for maximizing clubhead speed in a golf model.

Chapter 9

Visualization and Analysis Tools

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

If you cannot see it, you cannot understand it.

9.1 3D Motion Visualization

Visualization of multi-body trajectories is essential for understanding simulated motion and validating controller behavior. This section covers kinematics-based visualization (stick figures), manipulability analysis, and energy flow diagrams.

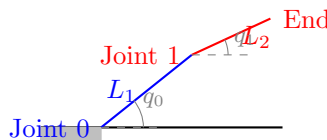


Figure 9.1: Stick-figure representation of a 2-link arm. Visualization code computes joint positions from angles using forward kinematics, then renders links and joints.

```
1 import numpy as np
2
3 def compute_joint_positions_2d(
4     q: np.ndarray,
5     lengths: list[float]
6 ) -> np.ndarray:
7     """Compute 2D joint positions for a serial chain.
8
9     Parameters
10    -----
11    q : np.ndarray, shape (n,)
12        Joint angles.
13    lengths : list[float]
```

```

14         Segment lengths.
15
16     Returns
17     -----
18     np.ndarray, shape (n+1, 2)
19         XY positions of each joint (including base).
20     """
21     n = len(q)
22     positions = np.zeros((n + 1, 2))
23     angle_cumulative = 0.0
24
25     for i in range(n):
26         angle_cumulative += q[i]
27         positions[i + 1, 0] = (positions[i, 0] +
28                               lengths[i] * np.cos(angle_cumulative))
29         positions[i + 1, 1] = (positions[i, 1] +
30                               lengths[i] * np.sin(angle_cumulative))
31
32     return positions
33
34
35 def phase_portrait(
36     q_history: np.ndarray,
37     qd_history: np.ndarray,
38     joint_idx: int = 0,
39     label: str = "Joint 0"
40 ) -> dict:
41     """Compute phase portrait data for a single joint.
42
43     Parameters
44     -----
45     q_history : (N, n) trajectory data
46     qd_history : (N, n) velocity data
47     joint_idx : int
48     label : str
49
50     Returns
51     -----
52     dict with 'x', 'y', 'label' keys
53     """
54     return {
55         'x': q_history[:, joint_idx],
56         'y': qd_history[:, joint_idx],
57         'label': label
58     }

```

Listing 9.1: Stick-figure animation for multi-body motion.

9.2 Jacobian and Manipulability Visualization

Visualizing the velocity manipulability ellipsoid JJ^T at each configuration [5]. The Jacobian maps joint velocities to end-effector velocities: $\mathbf{v}_{ee} = J(\mathbf{q})\dot{\mathbf{q}}$. The manipulability ellipsoid is the image of the unit ball under J , and its principal axes show the directions in which the end-effector can move with maximum ease.

9.3 Energy Flow Diagrams

Energy flow diagrams track how kinetic and potential energy change during motion, and how energy is transferred between segments via the kinetic chain. For a multi-segment system, the total energy is

$$E_{\text{total}} = \sum_i (KE_i + PE_i) = \sum_i \left(\frac{1}{2} I_i \dot{q}_i^2 + m_i g h_i \right) \quad (9.1)$$

where I_i is the effective inertia, $\dot{q}_i$ is the angular velocity, and h_i is the height of the center of mass of segment i . During ballistic phases (no muscle torque), energy is conserved and transferred between kinetic and potential forms.

Exercises

1. Animate a 3-link arm tracking a circular path.
2. Plot the manipulability ellipsoid during a reaching movement.
3. Create an energy flow diagram for the double pendulum golf model.

Chapter 10

Capstone: Golf Swing Analysis

Draft Notice. This chapter provides a runnable, tested three-link example and a roadmap for larger investigations. Its synthetic motion is prescribed, not measured or optimized. The extensions on muscles, feedback, uncertainty and impact require additional implementation and evidence.

The scientific question is how geometry, inertia and actuation together produce an end-point motion. A rapidly moving clubhead does not by itself reveal which joint supplied energy, which muscle was active, or what would happen after removing an input. Those questions require different calculations. This capstone connects them through one explicit mechanical model, then identifies the extra information each extension needs.

10.1 Model and Question

The implemented system has three rigid links and three revolute coordinates. The base is fixed. The first angle is measured counterclockwise from the horizontal; the other two are relative joint angles. World height increases upward and gravity acts downward. Angles, rates and accelerations use radians, rad/s and rad/s²; lengths, masses and generalized torques use metres, kilograms and Nm. The chain can illustrate a shoulder–elbow–wrist linkage, but it has no moving torso, two-arm closed loop, muscles, shaft flexibility, ground contact or ball collision.

Each link is a uniform rod with centre of mass at half its length and central inertia $I_i = m_i \ell_i^2 / 12$. The parallel-axis contributions are included when the model assembles its mass matrix. The default values are illustrative parameters, not subject-specific measurements. A head-heavy club requires measured or justified centre-of-mass and inertia parameters; changing its mass alone does not implement that distribution. The model therefore supports conditional mechanical deductions, not a prescription for human technique.

10.2 Geometry Before Attribution

Let $\theta_i = \sum_{j=1}^i q_j$ and $\omega_i = \sum_{j=1}^i \dot{q}_j$. The tip position and velocity are

$$\mathbf{p}(q) = \sum_{i=1}^3 \ell_i \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix}, \quad (10.1)$$

$$\mathbf{v}(q, \dot{q}) = \sum_{i=1}^3 \ell_i \omega_i \begin{bmatrix} -\sin \theta_i \\ \cos \theta_i \end{bmatrix} = J_p(q) \dot{q}. \quad (10.2)$$

Here J_p is the 2×3 Jacobian of the tip position. It is not the six-component spatial-twist Jacobian; the distinction matters when comparing robotics conventions [10]. Speed is $s_v = \|\mathbf{v}\|$, taken *after* adding the velocity vectors. Their directions depend on configuration. The scalar sum $\sum_i \ell_i |\omega_i|$ discards directional cancellation.

For the default lengths (0.28, 0.46, 1.10) m and relative rates (5, 5, 5) rad/s, the straight configuration $q = (0, 0, 0)$ gives 22.5 m/s. At $q = (0, \pi, \pi)$ the signed transverse contributions are $1.4 - 4.6 + 16.5 = 13.3$ m/s. The configuration-blind value would overestimate this particular folded example by about 69%. This is a reproducible geometric counterexample, not a population estimate.

10.3 What Adds at the Same State

For this unconstrained rigid model, write

$$M(q)\ddot{q} + b(q, \dot{q}) = \tau, \quad b = C(q, \dot{q})\dot{q} + g(q), \quad M(q) \succ 0. \quad (10.3)$$

Forward dynamics evaluates acceleration from the state and input; inverse dynamics evaluates the input required by a specified acceleration [9]. At a *common* state define

$$a_d = -M^{-1}b, \quad a_u = M^{-1}\tau, \quad \ddot{q} = a_d + a_u. \quad (10.4)$$

These are signed joint-acceleration vectors. The inverse is notation: the code solves linear systems. The equality is exact in the mathematical model, subject to numerical error in its implementation. It does not establish that arbitrary supplied position, velocity and torque arrays form a dynamically consistent trajectory.

Common Misconception

The magnitudes $\|a_d\|$ and $\|a_u\|$ generally do not add to $\|\ddot{q}\|$. If $\tau = b \neq 0$, then $a_u = -a_d$ and the net joint acceleration is zero, even though both component norms are positive. Reporting those norms as complementary percentages loses the cancellation. The norms also depend on the chosen joint coordinates and weighting. They are neither a speed partition nor a measure of muscular effort.

The tip acceleration contains an additional geometric term:

$$\mathbf{a}_p = J_p \ddot{q} + \dot{J}_p \dot{q} = \mathbf{a}_{p,d} + \mathbf{a}_{p,u}, \quad (10.5)$$

$$\mathbf{a}_{p,d} = J_p a_d + \dot{J}_p \dot{q}, \quad \mathbf{a}_{p,u} = J_p a_u, \quad (10.6)$$

$$\dot{J}_p \dot{q} = - \sum_{i=1}^3 \ell_i \omega_i^2 \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix}. \quad (10.7)$$

These vectors have units m/s². When $s_v > 0$, its instantaneous rate is

$$\dot{s}_v = \frac{\mathbf{v}^\top \mathbf{a}_{p,d}}{s_v} + \frac{\mathbf{v}^\top \mathbf{a}_{p,u}}{s_v}. \quad (10.8)$$

The two signed tangential terms can oppose one another. This formula is undefined at zero speed; acceleration vectors remain meaningful there. The current listing reports joint components, not these Cartesian projections.

At a straight vertical configuration, the ideal model has $g = 0$ and $C\dot{q} = 0$: derivatives of its relative-angle cosine terms vanish. Nevertheless, $\dot{J}_p\dot{q}$ can be nonzero. A rotating straight chain can have zero joint drift acceleration at that instant and a substantial centripetal tip acceleration. Straightness alone is insufficient even for zero joint drift: a horizontal chain generally has nonzero gravity torque.

10.4 A Counterfactual Is a New Trajectory

With $x = (q, \dot{q})$, write $\dot{x} = f(x) + G(x)\tau$. A zero-torque trajectory released at t_r obeys

$$\dot{x}_0 = f(x_0), \quad x_0(t_r) = x_u(t_r). \quad (10.9)$$

For a driven trajectory x_u , their difference satisfies

$$\frac{d}{dt}(x_u - x_0) = f(x_u) - f(x_0) + G(x_u)\tau. \quad (10.10)$$

The drift difference vanishes at release but generally not afterward. Thus integrating $M^{-1}\tau$ along the driven path does not by itself recover the difference in subsequent joint velocities. Different configurations also change the tip Jacobian. Control-affine input dependence does not make entire trajectories, or their speed magnitudes, superposable.

A useful comparison declares the release state, retained passive forces, intervention and endpoint: a common elapsed time and each trajectory's contact event are different experiments. Turning off excitation in a muscle model is also different from setting generalized torque to zero; activation and tendon states can retain history. Here the unforced system retains gravity and inertia and contains no friction or elastic elements. Energy conservation and timestep refinement provide checks on its numerical integration [8]. They do not validate its human anatomy. The separate `GolfModel.ztcf_trajectory` routine can support an unforced extension; `SwingAnalysis` does not call it.

10.5 A Reproducible Synthetic Example

The listing prescribes a smooth path, evaluates its analytic first and second derivatives, and calculates $\tau = M\ddot{q} + C\dot{q} + g$. These are required model torques for that path, with no imposed actuator limits. They replace an unrelated illustrative torque profile. Inverse-dynamic consistency tests an algebraic relationship; a separate forward integration and step-refinement comparison would test trajectory reproduction.

For duration $T > 0$ and maximum sample spacing $h > 0$, the grid contains $\lceil T/h \rceil + 1$ samples including both endpoints. Its actual spacing is $T/\lceil T/h \rceil$, which need not equal h . Analytic rates avoid confusing those two intervals. With $\sigma = t/T$, the prescribed path is

$$q(\sigma) = \begin{bmatrix} -\pi/2 + \pi\sigma^2 \\ (\pi/3)\sin(\pi\sigma) \\ -(\pi/2)\cos(\pi\sigma/2) \end{bmatrix}. \quad (10.11)$$

Its nonzero endpoint rates are allowed; this example is not a rest-to-rest trajectory. Doubling T at fixed path halves rates and quarters accelerations; gravity torque does not scale that way. This distinguishes changing the path, changing its timing and changing the mechanical plant.

```

1  """Same-state analysis of a prescribed three-link planar motion.
2
3  The generated Volume V listing uses this tested module. It computes tip speed
4  and instantaneous joint-acceleration components, not counterfactual
5  trajectories,
6  muscle forces or an optimized human swing. Synthetic torques are inverse-
7  dynamic
8  requirements of the prescribed motion under the uniform-rod model.
9  """
10
11 from __future__ import annotations
12
13 from dataclasses import dataclass, field
14 from typing import cast
15
16 import numpy as np
17 from numpy.typing import NDArray
18
19 from src.affine_control.golf_model import GolfModel
20
21 __all__ = ["SwingAnalysis", "synthetic_swing"]
22
23 type Array = NDArray[np.float64]
24
25 METRES_PER_SECOND_TO_MPH = 2.236936
26
27 def _rod_inertia(mass: float, length: float) -> float:
28     """Moment of inertia of a uniform rod about its own centre of mass."""
29     return mass * length**2 / 12.0
30
31
32 def _finite_array(value: Array, shape: tuple[int, ...], name: str) -> Array:
33     """Check shape and finiteness at the mutable analysis boundary."""
34     array = np.asarray(value, dtype=float)
35     if array.shape != shape or not np.all(np.isfinite(array)):
36         raise ValueError(f"{name} must be finite with shape {shape}")
37     return array
38
39
40 @dataclass
41 class SwingAnalysis:
42     """Kinematic and dynamic analysis of a three-link planar swing.
43
44     The chain is shoulder, elbow, wrist, carrying the club as the distal link
45     .
46     Angles are relative joint angles; the model treats each segment as a
47     uniform
48     rod, so its inertia follows from mass and length rather than being a free
49     parameter.
50     """
51
52     segment_lengths: Array = field(default_factory=lambda: np.array([0.28,
53     0.46, 1.10]))
54     segment_masses: Array = field(default_factory=lambda: np.array([2.16,
55     1.79, 0.30]))
56
57     time: Array | None = None
58     joint_angles: Array | None = None

```

```

55 joint_velocities: Array | None = None
56 joint_torques: Array | None = None
57
58 def model(self) -> GolfModel:
59     """The rigid-body model implied by the segment parameters."""
60     lengths_array = _finite_array(self.segment_lengths, (3,), "
segment_lengths")
61     masses_array = _finite_array(self.segment_masses, (3,), "
segment_masses")
62     if np.any(lengths_array <= 0):
63         raise ValueError("segment_lengths must be positive")
64     if np.any(masses_array <= 0):
65         raise ValueError("segment_masses must be positive")
66     lengths = tuple(map(float, lengths_array))
67     masses = tuple(map(float, masses_array))
68     inertias = tuple(_rod_inertia(m, ell) for m, ell in zip(masses,
lengths, strict=True))
69     return GolfModel(masses=masses, lengths=lengths, inertias=inertias)
70     # type: ignore[arg-type]
71
72 def _require_kinematics(self) -> tuple[Array, Array, Array]:
73     """Return '(time, angles, rates)', or fail with a usable message."
74     """
75     if self.time is None or self.joint_angles is None or self.
joint_velocities is None:
76         raise ValueError("set time, joint_angles and joint_velocities
before analysing")
77     time = np.asarray(self.time, dtype=float)
78     if time.ndim != 1 or time.size == 0 or not np.all(np.isfinite(time)):
79         raise ValueError("time must be a nonempty finite one-dimensional
array")
80     if np.any(np.diff(time) <= 0):
81         raise ValueError("time must be strictly increasing")
82     angles = _finite_array(self.joint_angles, (len(time), 3), "
joint_angles")
83     rates = _finite_array(self.joint_velocities, (len(time), 3), "
joint_velocities")
84     return time, angles, rates
85
86 def clubhead_speed(self) -> Array:
87     """Clubhead speed at each sample, from the tip Jacobian.
88
89     The tip velocity is 'sum_i qdot_i * (z_hat x r_i)', where 'r_i'
runs
90     from joint 'i' to the tip. Its magnitude depends on the
configuration
91     through the angles between those contributions -- which is precisely
what
92     a scalar sum of 'qdot_i * |r_i|' throws away.
93     """
94     _, angles, rates = self._require_kinematics()
95     model = self.model()
96     evaluate = np.vectorize(model.clubhead_speed, signature="(n),(n)->()")
97     return cast(Array, evaluate(angles, rates))
98
99 def joint_acceleration_components(self) -> tuple[Array, Array]:
100     """Return signed same-state drift/control arrays, each shape (N, 3).
    In relative-angle coordinates (rad/s^2), these are -M^-1(Cv+g) and

```

```

101         M-1 tau. Their sum is the model acceleration at each supplied state.
102         This algebra alone does not establish that input samples form a
103         solution.
104         """
105         _, angles, rates = self._require_kinematics()
106         if self.joint_torques is None:
107             raise ValueError("set joint_torques before decomposing")
108         torques = _finite_array(self.joint_torques, angles.shape, "
109         joint_torques")
110         model = self.model()
111
112         def evaluate(angle: Array, rate: Array, torque: Array) -> tuple[Array
113         , Array]:
114             """Solve both signed acceleration components at one state."""
115             mass = model.rigid_mass_matrix(angle)
116             bias = model.coriolis(angle, rate) @ rate + model.gravity_torque(
117             angle)
118             return np.linalg.solve(mass, -bias), np.linalg.solve(mass, torque
119             )
120
121         batch = np.vectorize(evaluate, signature="(n),(n),(n)->(n),(n)")
122         return cast(tuple[Array, Array], batch(angles, rates, torques))
123
124     def ztcf_decomposition(self) -> tuple[Array, Array]:
125         """Return same-state joint-acceleration norms in rad/s2.
126
127         The historical name is retained, but no trajectory is integrated.
128         These
129         Euclidean norms do not add to net acceleration norm, do not measure
130         tip
131         acceleration, and do not partition clubhead speed or causal
132         importance.
133         """
134         drift, control = self.joint_acceleration_components()
135         return np.linalg.norm(drift, axis=1), np.linalg.norm(control, axis=1)
136
137     def summary(self) -> str:
138         """Human-readable summary of the analysis."""
139         time, _, _ = self._require_kinematics()
140         speeds = self.clubhead_speed()
141         peak = float(np.max(speeds))
142         peak_index = int(np.argmax(speeds))
143
144         lines = [
145             "=== Golf Swing Analysis Summary ===",
146             f"Duration: {time[-1] - time[0]:.3f} s",
147             f"Peak clubhead speed: {peak:.1f} m/s ({peak *
148             METRES_PER_SECOND_TO_MPH:.1f} mph)",
149             f"Peak speed time: {time[peak_index]:.3f} s",
150         ]
151
152         if self.joint_torques is not None:
153             drift, control = self.ztcf_decomposition()
154             lines.append(
155                 "Joint acceleration norm medians (rad/s2): "
156                 f"drift {np.median(drift):.1f}, control {np.median(control)
157                 :.1f}"
158             )
159         return "\n".join(lines)

```

```

151
152 def _prescribed_motion(time: Array, duration: float) -> tuple[Array, Array,
    Array]:
153     """Evaluate the analytic angles and their first two time derivatives."""
154     fraction = time / duration
155     angles = np.column_stack(
156         [
157             -np.pi / 2 + np.pi * fraction**2,
158             np.pi / 3 * np.sin(np.pi * fraction),
159             -np.pi / 2 * np.cos(np.pi * fraction / 2),
160         ]
161     )
162     rates = (
163         np.column_stack(
164             [
165                 2 * np.pi * fraction,
166                 np.pi**2 / 3 * np.cos(np.pi * fraction),
167                 np.pi**2 / 4 * np.sin(np.pi * fraction / 2),
168             ]
169         )
170         / duration
171     )
172     accelerations = (
173         np.column_stack(
174             [
175                 np.full_like(fraction, 2 * np.pi),
176                 -np.pi**3 / 3 * np.sin(np.pi * fraction),
177                 np.pi**3 / 8 * np.cos(np.pi * fraction / 2),
178             ]
179         )
180         / duration**2
181     )
182     return angles, rates, accelerations
183
184
185 def synthetic_swing(duration: float = 0.30, dt: float = 0.001) ->
    SwingAnalysis:
186     """Return prescribed motion with matching model inverse-dynamics torques.
187
188     Positive finite duration and dt are in seconds; dt is the maximum sample
189     spacing, with both endpoints included. Analytic derivatives avoid a
190     mismatch
191     between grid spacing and numerical differentiation. Torques satisfy the
192     model
193     equations at these samples (up to model/numerical error); this is neither
194     a
195     forward-integration accuracy test nor measured or optimized human motion.
196     """
197     if not np.isfinite(duration) or duration <= 0:
198         raise ValueError("duration must be finite and positive")
199     if not np.isfinite(dt) or dt <= 0:
200         raise ValueError("dt must be finite and positive")
201     samples = int(np.ceil(duration / dt)) + 1
202     time = np.linspace(0.0, duration, samples)
203     angles, rates, accelerations = _prescribed_motion(time, duration)
204     analysis = SwingAnalysis(time=time, joint_angles=angles, joint_velocities
        =rates)
205     model = analysis.model()
206
207     def inverse(angle: Array, rate: Array, acceleration: Array) -> Array:

```

```

205     """Return generalized torque required for one prescribed state."""
206     return (
207         model.rigid_mass_matrix(angle) @ acceleration
208         + model.coriolis(angle, rate) @ rate
209         + model.gravity_torque(angle)
210     )
211
212     batch = np.vectorize(inverse, signature="(n),(n),(n)->(n)")
213     analysis.joint_torques = batch(angles, rates, accelerations)
214     return analysis

```

Listing 10.1: Prescribed-Motion Analysis, generated from `src/affine_control/swing_analysis.py`.

The listing is generated from the canonical module and checked for freshness. Run it with:

```

1 from src.affine_control.swing_analysis import synthetic_swing
2
3 print(synthetic_swing().summary())

```

Listing 10.2: Running the Prescribed-Motion Analysis

The summary reports duration, peak tip speed and its timestamp, plus separate medians of joint-acceleration norms in rad/s^2 . The peak is among sampled values, not a certified continuous-time maximum or a ball-impact value. The medians are descriptive sample statistics and use every supplied sample; nonuniform sampling would weight densely sampled periods more heavily. The historical method name `ztcf_decomposition` returns those norms; `joint_acceleration_components` exposes the signed vectors. Neither method integrates a counterfactual.

10.6 Connecting Mechanics, Control and Measurement

This example implements model construction, prescribed-motion inverse dynamics, tip kinematics and same-state joint decomposition. Optimization, feedback, robustness certificates, muscle estimation and variability analysis are *extensions*, each with a distinct evidence requirement.

For experimental inverse dynamics, start with synchronized measurements and a model that specifies coordinates, inertial parameters and external wrenches. Choose sampling and filtering from the signal bandwidth, noise and derivative accuracy needed; there is no universal adequate camera rate. In a whole-body model, unmeasured contact forces cannot simply be omitted without changing the inference. Report assumed forces, constraints and residuals, especially in unactuated base coordinates. Motion alone does not generally identify all external forces and internal torques simultaneously.

Net joint torque also does not uniquely specify muscle force. Moment arms, activation and contraction dynamics, passive tissues, force bounds and the chosen recruitment criterion affect that estimate. EMG can constrain an activation model; it is not a direct force or torque measurement. Refer to the biological actuation and inverse-dynamics treatments in Volume III before interpreting recruitment. A later torque response can also depend on earlier control through the current velocity, geometry and stored elastic energy.

For the ideal conservative rigid model, total mechanical energy satisfies $\dot{E} = \dot{q}^T \tau$. Internal inertial coupling redistributes motion and energy; it is not an additional energy supply.

A joint can have a large acceleration contribution while doing little instantaneous work if its rate is small. Peak ordering, acceleration attribution and joint power therefore answer different questions. Specify the system boundary before interpreting energy transferred across it.

A feedback study adds a declared policy, disturbances, sensor information and actuator constraints. A finite-time funnel must bound a set of initial states through an interval and into a defined terminal set; a calculation only at the nominal impact instant is insufficient. If contact is included, the guard and reset map must be part of the argument. A contraction condition or successful simulation should be reported only over the domain where its hypotheses or numerical checks hold.

For variability, an ensemble and a task map are essential. At a reference state, Dh separates first-order task-changing directions from its null space. Variance depends on alignment, coordinates, metric, rank and normalization by subspace dimension. Artificial torque perturbations describe the simulated ensemble; they do not establish a human neural synergy. If the task is a peak over time, its differentiability and event alignment require care. A fixed-time endpoint map is a simpler starting point.

Final Exercises

1. **Kinematic and Dynamic Consistency.** Run the example and compare the two configurations above. Verify analytic endpoint rates and the residual $M\ddot{q} + C\dot{q} + g - \tau$. Explain why the latter does not independently validate the model or its forward integrator.
2. **Two Parameter Studies.** First hold the prescribed joint motion fixed and vary club length and mass separately. Mass changes required torque but not tip kinematics; length changes both. Then construct a forward study with the same initial state and declared input policy. Compare outcomes and explain why these are different experiments.
3. **Signed Contributions.** Set $\tau = b$ at a chosen state. Check cancellation in joint space and compute the remaining $\dot{J}_p \dot{q}$ tip acceleration. Contrast vector, norm, tangential-acceleration and power measures.
4. **Release Experiment.** Integrate driven and zero-torque systems from a shared state over a declared interval. Refine the timestep, monitor energy where appropriate, and compare endpoint velocity vectors as well as speeds. Repeat for a different release time without treating either result as a universal contribution percentage.
5. **Muscle and Robustness Extensions.** Specify a muscle model, recruitment objective and feasibility checks, or a feedback policy and terminal set. Implement one bounded extension and report which conclusions depend on its assumptions. An impact study must define contact and reset behavior.
6. **Variability Extension.** Generate an ensemble under an explicit noise model, align samples and analyze a fixed-time task Jacobian. Report sample-size sensitivity and dimension-normalized variances. Explain what additional data would be needed to make a claim about human coordination.

Bibliography

- [1] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955. Legacy duplicate key retained for backwards compatibility.
- [2] R. Featherstone. The calculation of robot dynamics using articulated-body inertias. *International Journal of Robotics Research*, 2(1):13–30, 1983.
- [3] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.
- [4] D. H. Jacobson and D. Q. Mayne. *Differential Dynamic Programming*. American Elsevier Publishing Company, 1970.
- [5] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.
- [6] W. Lohmiller and J.-J. E. Slotine. On contraction analysis for non-linear systems. *Automatica*, 34(6):683–696, 1998.
- [7] J. Y. S. Luh, M. W. Walker, and R. P. C. Paul. On-line computational scheme for mechanical manipulators. *Journal of Dynamic Systems, Measurement, and Control*, 102(2):69–76, 1980. Legacy duplicate key retained for backwards compatibility.
- [8] Kevin M. Lynch and Frank C. Park. Modern robotics: Forward dynamics of open chains. Northwestern University, video supplement and transcript. Accessed 2026-10-05. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/8-5-forward-dynamics-of-open-chains/>.
- [9] Kevin M. Lynch and Frank C. Park. Modern robotics: Lagrangian formulation of dynamics, part 1. Northwestern University, video supplement and transcript. Accessed 2026-10-05. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/chapter-8-1-lagrangian-formulation-of-dynamics-part-1-of-2/>.
- [10] Kevin M. Lynch and Frank C. Park. Modern robotics: Space jacobian. Northwestern University, video supplement and transcript. Accessed 2026-10-05. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/5-1-1-space-jacobian/>.
- [11] A. Majumdar and R. Tedrake. Funnel libraries for real-time robust feedback motion planning. *The International Journal of Robotics Research*, 36(8):947–982, 2017.
- [12] I. R. Manchester and J.-J. E. Slotine. Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design. *IEEE Transactions on Automatic Control*, 62(6):3046–3053, 2017.
- [13] D. Q. Mayne. A second-order gradient method for determining optimal trajectories of non-linear discrete-time systems. *International Journal of Control*, 3(1):85–95, 1966.

-
- [14] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. Legacy duplicate key retained for backwards compatibility.
 - [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. Legacy duplicate key retained for backwards compatibility.
 - [16] J.-J. E. Slotine and W. Li. *Applied Nonlinear Control*. Prentice Hall, 1991.
 - [17] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
 - [18] Russ Tedrake. *Underactuated Robotics: Algorithms and Control*. MIT OpenCourseWare, 2021. <http://underactuated.mit.edu>.
 - [19] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. Legacy duplicate key retained for backwards compatibility.